DMBI Module 4

DBSCAN Algorithm

DBSCAN is an unsupervised clustering algorithm that groups data points based on density and identifies outliers as noise.

Key Concepts

- Core Idea: Clusters form where points are closely packed; points in low-density regions become noise.
- Parameters:
  - `ε` (epsilon): Radius of the neighbourhood around a point.
  - `MinPts`: Minimum number of points required within ε to form a dense region.
- Point Types:
  - Core Point: Has at least `MinPts` neighbours within `ε`.
  - Border Point: Within ε of a core point but not dense enough itself.
  - Noise Point: Neither core nor border (outlier).

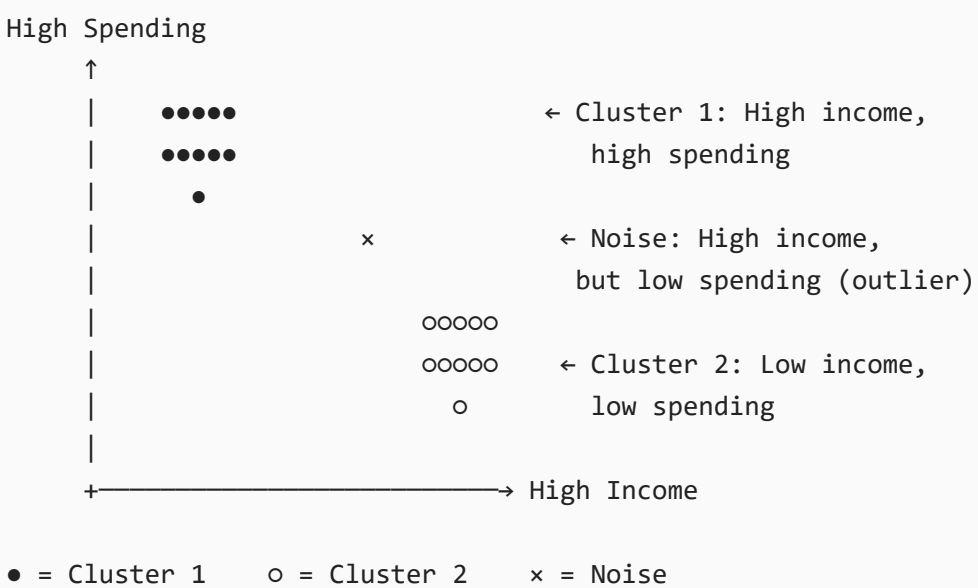
Advantages & Limitations

Advantages	**Limitations**
No need to predefine `k` (unlike KMeans)	Sensitive to `ε` and `MinPts` selection
Handles arbitrary-shaped clusters	Struggles with varying density
Detects noise/outliers automatically	

Working Steps

1. Select an unvisited point.
2. Find all points within ε distance.
3. If `neighbors ≥ MinPts` → mark as core point, form a new cluster, expand by adding all density-reachable points.
4. If not a `core point → mark as noise` (may later become border if connected).
5. Repeat until all points are processed.

Diagram



Example: Customer Data (Income vs Spending Score)
Parameters: `ε = 3, MinPts = 4`
Cluster 1: - High-income, high-spending customers
Cluster 2: - Low-income, low-spending customers
Noise: - Irregular customers with unusual spending patterns (High income but very low spending)

K-Means Algorithm

K-Means is an unsupervised machine learning algorithm used to partition a dataset into `K` distinct, non-overlapping clusters based on similarity. It groups data points such that points within the same cluster are as close as possible (high intra-cluster similarity) while points in different clusters are as far as possible (low inter-cluster similarity). The "K" represents the number of clusters, which must be specified in advance.

How K-Means Works (Step-by-Step)

1. Choose `K`: Decide the number of clusters you want.
2. Initialize Centroids: Randomly select `K` data points as initial cluster centres (centroids).
3. Assign Points: For each data point, calculate its distance to each centroid (usually Euclidean distance). Assign the point to the cluster whose centroid is nearest.
4. Update Centroids: After all points are assigned, recalculate each centroid as the mean (average) of all points in that cluster.

5. Repeat: Steps 3 and 4 are repeated until convergence — when centroids no longer move significantly or assignments stop changing.

Advantages of K-Means:

- Simple & Fast: Easy to understand and implement
- Scalable: Works efficiently on large datasets `(O(n × K × iterations))`
- Guaranteed Convergence: Always converges (though possibly to a local minimum)

Limitations of K-Means:

- Requires `K` in advance: Often unknown for real-world data
- Sensitive to initialization: Poor centroid seeds lead to bad clusters `(use K-Means++)`
- Assumes spherical clusters: Fails for non-convex or irregular shapes
- Affected by outliers: Outliers skew centroid positions
- Assumes equal cluster sizes: Performs poorly when clusters have very different densities

---

Hierarchical clustering

Agglomerative Hierarchical Clustering (Bottom-Up Approach)

1. Start with N clusters: Treat each data point as its own individual cluster (`N` clusters for `N` points)
2. Compute distance matrix: Calculate the distance between every pair of clusters using a chosen distance metric (Euclidean, Manhattan, etc.)
3. Find closest clusters: Identify the pair of clusters with the smallest distance between them
4. Merge clusters: Combine the two closest clusters into a single new cluster
5. Update distance matrix: Recalculate distances between the new cluster and all remaining clusters using a linkage criterion:
    - Single linkage: Minimum distance between points in two clusters
    - Complete linkage: Maximum distance between points in two clusters
    - Average linkage: Average distance between all pairs of points
    - Ward's linkage: Minimize variance within clusters
6. Repeat: Continue steps 3-5 until only one cluster remains
7. Form dendrogram: Record each merge to create a tree-like diagram (dendrogram)
8. Cut the dendrogram: Choose a height threshold to decide the final number of clusters

Divisive Hierarchical Clustering (Top-Down Approach):

1. Start with one cluster: Place all data points in a single cluster

2. Split the cluster: Divide the cluster into two most dissimilar sub-clusters
3. Repeat recursively: Continue splitting each sub-cluster until each point is its own cluster or a stopping criterion is met
4. Form dendrogram: Record each split to create the tree structure

Example
Data points: A, B, C, D (with distances: `A-B=2, A-C=6, A-D=10, B-C=5, B-D=9, C-D=3`)

Steps:

1. Start: `{A}, {B}, {C}, {D}`
2. Merge `A-B` (distance 2) → `{A,B}, {C}, {D}`
3. Merge `C-D` (distance 3) → `{A,B}, {C,D}`
4. Merge `{A,B}` and `{C,D}` (distance between clusters) → `{A,B,C,D}`

Dendrogram:

BIRCH Algorithm

`BIRCH` (Balanced Iterative Reducing and Clustering using Hierarchies) is a clustering algorithm designed for large datasets. It incrementally builds a compact summary of the data using a `CF` (Clustering Feature) tree, then applies clustering on this summary. It is efficient, memory-friendly, and handles noise well.

CF Tree (Clustering Feature Tree):
A hierarchical data structure that stores compact summaries of data points. Each node contains clustering features: number of points, linear sum, and square sum.

Phases:

- Phase 1: Build `CF` tree by inserting data points.
- Phase 2: Condense the tree by removing outliers and noise.
- Phase 3: Apply global clustering (e.g., k-means) on leaf entries.
- Phase 4 (optional): Refine clusters further.

Advantages

- Handles very large datasets efficiently.
- Requires only one database scan in most cases.
- Robust against noise.

Example:
Suppose we have 10,000 customer transaction records. Instead of clustering all 10,000 directly, BIRCH builds a `CF` tree summarizing them into, say, 100 compact nodes.
Then clustering (e.g., k-means) is applied on these 100 nodes instead of the full dataset. The result is faster clustering with similar accuracy, even under memory constraints.

Explain Multilevel and Multidimensional Association rules with suitable examples.

Multilevel Association Rules:

- Definition: Rules involving two or more distinct levels of abstraction within a concept hierarchy, where items are organized from general categories to specific instances.
- Concept: They combine multiple hierarchical levels to form patterns that span from broad to granular perspectives.
- Goal: Discover relationships across different abstraction levels, from general categories down to specific products.
- Example: Age = Young `AND` Income = High → Buys = Electronics (Level 1) `OR` Buys = Laptop (Level 2) `OR` Buys = Gaming Laptop (Level 3).
- Application: Targeted marketing campaigns based on product category preferences at different levels of specificity combined with demographic data.

Multidimensional Association Rules:

- Definition: Rules involving two or more distinct attributes (dimensions) such as age, income, or location.
- Concept: They combine multiple predicates to form richer patterns.
- Goal: Discover complex relationships across different data dimensions. Example: Age = Young `AND` Income = High → Buys = Luxury Car.
- Application: Targeted marketing campaigns based on demographic and behavioral data combined.

Explain Market Basket Analysis with example

- Market Basket Analysis is a data mining technique used to discover relationships between items that customers frequently purchase together in a single transaction.
- It identifies association rules of the form "If a customer buys product `X`, they are likely to also buy product `Y`."
- The technique derives its name from the shopping cart (basket) that customers fill with items during a store visit.
- The most common algorithm used for Market Basket Analysis is `Apriori`, which generates rules based on metrics like support, confidence, and lift.

Key Metrics:

- Support: The percentage of transactions containing a specific item or itemset. Example: `Support of Bread = 40% means 40%` of all transactions include Bread.
- Confidence: The conditional probability that a customer buys `Y` given that they bought `X`. Example: Confidence of `Milk → Bread = 60%` means `60%` of customers who buy Milk also buy Bread.

Example
Consider a small grocery store with five transactions:

Transaction 1: Bread, Milk, Eggs
Transaction 2: Bread, Butter
Transaction 3: Bread, Milk, Butter
Transaction 4: Milk, Eggs
Transaction 5: Bread, Milk, Butter, Eggs

Analyzing the rule Bread → Milk:

- Total transactions = `5`
- Transactions containing `Bread = 4 (T1, T2, T3, T5) → Support(Bread) = 80%`
- Transactions containing both `Bread and Milk = 3 (T1, T3, T5) → Support(Bread ∪ Milk) = 60%`
- Confidence = Support(Both) / Support(Bread) = `60% / 80%` = `75%`
- Interpretation: `75%` of customers who buy Bread also buy Milk.

How to Improve Efficiency of Apriori Algorithm?

- Hash-Based Technique: Use hash tables to reduce candidate `2-itemsets`. When generating pairs, store counts in a hash bucket and remove candidates with low hash

counts before scanning the database.

- Partitioning: Divide the database into smaller blocks that fit in memory. Mine each partition independently for local frequent `itemsets`, take their union as global candidates, then scan the full database once for verification. This requires only two total scans.
- Transaction Reduction: Remove transactions that no longer contain any frequent `itemsets` in later passes. A transaction with fewer than k items cannot support a `k-itemset`, eliminating it from higher-level scans.
- Dynamic Itemset Counting: Add new candidate `itemsets` during the database scan rather than before it. When an itemset becomes frequent, immediately generate its supersets, reducing the number of passes.
- Best Alternative: Replace `Apriori` entirely with `FP-Growth`, which compresses data into an `FP-tree` and mines patterns without candidate generation, making it 2-3 times faster.